

SUBMISSION CHECKLIST

OptiCrop Executable Files & Deployment Guide

Complete submission inventory, file structure overview, deployment configuration, and step-by-step run instructions for the OptiCrop agricultural optimization engine.

Team Lead: Peddehapu Niharika

Members: Polupalli Sunil | Romeo Mhakayakora | Sk Salomi | Jotshna Aditya Tadi

Version 1.0 | July 2026

Table of Contents

1. Submission Checklist	2
2. File and Folder Structure	3
Directory Purpose Summary	4
3. Deployment Details	5
Deployment Configuration	5
Repository Reference	6
Environment Variables	6
Cloud Database Considerations	6
Access URL	7
4. Run and Download Instructions	7
Option A: Running from the Source Archive (Manual Setup)	7
Option B: Running via Automated Bootstrap Scripts	8
Windows Users (setup.bat)	8
Linux / macOS Users (setup.sh)	8
Option C: Downloading and Installing the Android APK (Offline Farmers)	9
5. Known Issues and Workarounds	10
Appendix: Quick Reference Card	11
API Endpoint Summary	11
Key File Locations	12
Team Contact Matrix	12

1. Submission Checklist

The following table provides a comprehensive inventory of all deliverable items submitted as part of the OptiCrop project. Each item is verified against the project's submission requirements and marked with its completion status.

#	Deliverable Item	Description	Status
1	Source Code — Backend	Complete Flask application (<code>app.py</code>) implementing input validation, model inference via <code>joblib</code> , prediction serving on the <code>/predict</code> route, and database logging on the <code>/log</code> route.	✓ Completed
2	Source Code — Frontend	Responsive HTML/CSS/JS interface (<code>templates/index.html</code>) with card-based input forms for all seven agricultural parameters, asynchronous fetch-based prediction requests, and dynamic result rendering.	✓ Completed
3	README File	Comprehensive project readme (<code>README.md</code>) containing project overview, feature list, installation instructions, usage guide, API endpoint documentation, and contributor credits.	✓ Completed
4	Dependency List	Python package dependencies (<code>requirements.txt</code>) specifying exact versions of Flask, scikit-learn, joblib, numpy, pandas, and other required libraries.	✓ Completed
5	Serialized ML Model	Pre-trained Random Forest pipeline (<code>model/crop_recommendation_model.pkl</code>) serialized via <code>joblib.dump()</code> , containing the classifier, StandardScaler, and preprocessing transformers.	✓ Completed
6	Example Environment File	Template environment configuration (<code>.env.example</code>) listing all configurable environment variables (database path, host port, debug mode) with placeholder values.	✓ Completed
7	Hosted Web URL	Cloud-deployed application accessible at the Render hosting platform URL, providing real-time predictions through a public browser interface.	✓ Completed
8	Database Schema	Complete SQLite schema definition including table creation statements, foreign key constraints, and the Entity Relationship Diagram (ERD) for Users, Soil Metrics, Climate Metrics, and Recommendations.	✓ Completed
9	Demo Video	Screen-capture demonstration video (MP4 format, under 5 minutes) walking through the complete user workflow: entering soil-climate parameters, receiving a crop recommendation, and viewing the result card.	✓ Completed

1 0	Project Documentation	Detailed project documentation (<code>Project_Documentation.md</code>) covering executive summary, scenarios, architecture, ML pipeline, database schema, implementation details, distribution strategy, and epic milestones.	✓ Completed
1 1 1	Bootstrap Script (Windows)	Automated setup script (<code>setup.bat</code>) for Windows environments that creates a virtual environment, installs dependencies, and launches the Flask server in a single click.	✓ Completed
1 2	Bootstrap Script (Linux)	Automated setup script (<code>setup.sh</code>) for Linux/macOS environments performing the equivalent bootstrap operations with error handling and diagnostics.	✓ Completed
1 3	Android APK (Offline)	Pre-compiled Android Package Kit (<code>opti-crop.apk</code>) embedding the ML model and local SQLite cache for fully offline crop prediction on Android 7.0+ devices.	✓ Completed
1 4	Static Assets	CSS stylesheets, JavaScript files, and image assets (<code>static/</code> directory) supporting the frontend interface's visual design, responsive layout, and interactive behavior.	✓ Completed
1 5	Database Seed File	Initial seed data (<code>seed_data.sql</code>) containing the crop reference database with ideal parameter ranges for all 22 supported crop types, used to populate the local SQLite cache.	✓ Completed

Table 1: Submission Checklist — Deliverable Items

2. File and Folder Structure

The OptiCrop project follows a clean, modular directory layout that separates source code, machine learning artifacts, static assets, templates, and configuration files into logically organized directories. This structure supports both local development and cloud deployment without modification.

```
opti-crop/
|
|-- app.py # Flask application entry point
|-- requirements.txt # Python dependency specification
|-- README.md # Project overview and documentation
|-- .env.example # Template environment variable configuration
|-- .gitignore # Git exclusion rules
|-- setup.bat # Windows automated bootstrap script
|-- setup.sh # Linux/macOS automated bootstrap script
|-- seed_data.sql # SQLite seed data for crop reference
|
|-- model/
|   |-- crop_recommendation_model.pkl # Serialized ML pipeline
|   |
|   |-- templates/
|   |   |-- index.html # Main application UI
|   |   |
|   |-- static/
|   |   |-- css/
|   |   |   |-- style.css # Responsive layout stylesheet
|   |   |-- js/
|   |   |   |-- main.js # Frontend JavaScript
|   |   |   |-- images/
|   |   |   |-- logo.png # OptiCrop project logo
|   |   |
|   |-- database/
|   |   |-- opti_crop.db # SQLite database (runtime, not committed)
|   |   |-- schema.sql # Table creation statements
|   |   |
|   |-- apk/
|   |   |-- opti-crop.apk # Pre-compiled Android APK
|   |   |-- apk_build_notes.md # APK build configuration
|   |   |
|   |-- docs/
|   |   |-- Project_Documentation.md # Comprehensive project documentation
|   |   |-- Project_Executable_Files.md # Submission checklist
|   |   |
|   |-- notebooks/
|   |   |-- eda_analysis.ipynb # Exploratory Data Analysis notebook
|   |   |-- model_training.ipynb # Model training and evaluation notebook
|   |   |   |-- results/
|   |   |   |-- feature_importance.png
|   |   |   |-- confusion_matrix.png
|   |   |   |-- classification_report.txt
|   |   |
|   |-- tests/
|   |   |-- test_app.py # Unit tests for Flask routes
|   |   |-- test_model.py # Unit tests for model inference
|   |   |-- conftest.py # Pytest fixtures
```

Figure 1: OptiCrop Project Directory Tree

Directory Purpose Summary

Directory	Purpose
model/	Stores the serialized machine learning model artifact produced during training. This is the only file from the ML pipeline that is required at runtime.

templates/	Contains the Jinja2 HTML template served by Flask. The single-file architecture (<code>index.html</code>) keeps the frontend simple and self-contained.
static/	Houses CSS, JavaScript, and image assets referenced by the HTML template. Organized by type for clarity.
database/	Contains the SQLite database file (generated at runtime) and the SQL schema definition used to create it.
apk/	Holds the pre-built Android APK and its build notes. This directory is only relevant for the offline distribution channel.
docs/	Project documentation files — the primary reference for understanding the system's design, architecture, and usage.
notebooks/	Development-time Jupyter notebooks used for EDA, model training, and result analysis. Not required at runtime but included for reproducibility.
tests/	Automated test suite covering backend route behavior, input validation, and model inference correctness. Run via <code>pytest</code> .

Table 2: Directory Purpose Summary

3. Deployment Details

OptiCrop's cloud-hosted web version is deployed on the **Render** platform, which provides a fully managed cloud environment for Python web applications. Render was selected for its straightforward Git-based deployment workflow, its free tier (suitable for academic and demonstration purposes), and its native support for Python WSGI applications via Gunicorn.

Deployment Configuration

Configuration Item	Value / Description
Platform	Render — Web Service
Environment	Python 3.10.x
Build Command	<code>pip install -r requirements.txt</code>
Start Command	<code>gunicorn app:app</code>
Instance Type	Free tier (512 MB RAM, 0.1 CPU) — upgradeable to Starter or Standard for production workloads
Region	Singapore (closest to the primary user base in India, minimizing latency)
Branch	<code>main</code> — auto-deploys on every push
Health Check Path	<code>/</code> — Render pings this path to verify the service is responsive

Table 3: Deployment Configuration

Repository Reference

Item	Details
Version Control	Git (GitHub / GitLab)
Repository Name	opti-crop
Default Branch	main
Webhook Integration	Render is connected to the repository via a deploy webhook; every push to <code>main</code> triggers an automatic rebuild and redeploy.

Table 4: Repository Reference

Environment Variables

The following environment variables are configured in the Render dashboard under the service's "Environment" section. The corresponding `.env.example` file in the project root provides a template for local development.

Variable Name	Purpose	Example Value
FLASK_ENV	Sets the Flask runtime environment mode	production
SECRET_KEY	Used for session management and CSRF protection in the cloud version	*(32-character random string)*
DATABASE_PATH	Absolute file path for the SQLite database on the Render filesystem	/opti-crop/database/opti_crop.db
MODEL_PATH	Absolute file path for the serialized ML model	/opti-crop/model/crop_recommendation_model.pkl
DEBUG	Enables or disables Flask debug mode (must be <code>False</code> in production)	False
HOST	Binding address for the WSGI server	0.0.0.0
PORT	Binding port (Render injects this automatically via <code>PORT</code> env var)	*(auto-assigned by Render)*

Table 5: Environment Variables

Cloud Database Considerations

In the current deployment, the SQLite database file is stored on the Render instance's ephemeral filesystem. This is sufficient for demonstration and low-traffic research use cases. For production deployments requiring persistent, scalable storage, the database can be migrated to an AWS RDS instance (PostgreSQL or MySQL) by updating the `DATABASE_PATH` environment variable and modifying the database connection logic to use SQLAlchemy with the appropriate database driver. The schema translation from SQLite to PostgreSQL is straightforward, as the table definitions use standard SQL types that are compatible across both database engines.

Access URL

The deployed application is accessible at the following URL (placeholder — replace with the actual Render-assigned subdomain):

Access Type	URL
Web Application	<code>https://opti-crop.onrender.com</code>
Prediction API	<code>https://opti-crop.onrender.com/predict</code> (POST, JSON body)
Health Check	<code>https://opti-crop.onrender.com/</code> (GET, returns 200 OK)

Table 6: Access URLs

4. Run and Download Instructions

This section provides step-by-step instructions for three distinct usage scenarios: running the application locally from the source archive, running it via the automated bootstrap scripts, and installing the offline Android APK on a mobile device.

Option A: Running from the Source Archive (Manual Setup)

This method is intended for developers and technically proficient users who want full control over the runtime environment.

Step 1: Extract the Archive

Download the project archive (`opti-crop.zip` or `opti-crop.tar.gz`) and extract its contents to a directory of your choice. On Windows, right-click the archive and select "Extract All." On Linux or macOS, open a terminal and run the extraction command appropriate to the archive format.

Step 2: Verify Python Installation

Ensure that Python 3.8 or a later version is installed on your system. Open a terminal or command prompt and run the Python version check command. If Python is not installed, download it from the official Python website (python.org) or install it via your operating system's package manager.

Step 3: Create a Virtual Environment

Navigate to the extracted project directory. Create a Python virtual environment named `venv` using the built-in `venv` module. This isolates the project's dependencies from your system-wide Python installation, preventing version conflicts.

Step 4: Activate the Virtual Environment

- On Windows: run the activation script located at `venv\Scripts\activate`
- On Linux/macOS: run the activation script located at `venv/bin/activate`

After activation, your terminal prompt will be prefixed with `(venv)` to indicate that the virtual environment is active.

Step 5: Install Dependencies

With the virtual environment active, install all required Python packages by running the `pip install` command against the `requirements.txt` file. This will download and install Flask, scikit-learn, joblib, numpy, pandas, and all other listed dependencies.

Step 6: Verify the Model File

Confirm that the serialized model file exists at `model/crop_recommendation_model.pkl`. This file is essential for prediction; the application will fail to start if it is missing.

Step 7: Launch the Application

Run the Flask application using Python. The server will start on `http://127.0.0.1:5000` by default. Open this URL in your web browser to access the OptiCrop interface.

Option B: Running via Automated Bootstrap Scripts

This method is recommended for non-technical users, agricultural extension staff, and workshop participants who need a one-click setup experience.

Windows Users (`setup.bat`)

- 1. Extract the project archive to any directory on your Windows machine.
- 2. Ensure Python 3.8+ is installed and added to your system PATH.
- 3. Double-click the `setup.bat` file in the project root directory.
- 4. A command prompt window will open and display progress messages as the script creates the virtual environment, installs dependencies, and starts the server.
- 5. When the script displays "Server running at `http://127.0.0.1:5000`", your default web browser will open automatically to the OptiCrop interface.
- 6. To stop the server, close the command prompt window or press `Ctrl+C` in the terminal.

Linux / macOS Users (`setup.sh`)

- **1.** Extract the project archive to any directory on your Linux or macOS machine.
- **2.** Ensure Python 3.8+ is installed (on Ubuntu/Debian: `sudo apt install python3 python3-pip python3-venv`).
- **3.** Open a terminal and navigate to the extracted project directory.
- **4.** Run the bootstrap script by executing `chmod +x setup.sh` followed by `./setup.sh`.
- **5.** The script will display progress messages, create the virtual environment, install dependencies, and launch the server.
- **6.** Open `http://127.0.0.1:5000` in your web browser to access the OptiCrop interface.
- **7.** To stop the server, press `Ctrl+C` in the terminal.

Option C: Downloading and Installing the Android APK (Offline Farmers)

This method is designed for farmers and field workers who need to use OptiCrop on an Android smartphone without internet connectivity.

Step 1: Obtain the APK File

The `opti-crop.apk` file can be obtained through any of the following channels:

- **Direct file transfer:** Receive the APK on a microSD card or USB flash drive from an agricultural extension officer or NGO distribution point.
- **Bluetooth transfer:** Receive the APK via Bluetooth from another farmer's device that already has the file.
- **WhatsApp / messaging app:** Receive the APK as a file attachment through any messaging application that supports file sharing.
- **USB sideloading:** Connect the Android device to a computer via USB cable and copy the APK to the device's "Downloads" folder.

Step 2: Enable "Install from Unknown Sources"

Before installing the APK, the Android device must be configured to allow installation from sources other than the Google Play Store. Navigate to the device's Settings menu, select "Security" (or "Privacy" on newer Android versions), and enable the "Allow installation from unknown sources" or "Install unknown apps" toggle. On Android 8.0 and above, this permission is granted per-app — select the file manager or browser app you will use to open the APK and grant it installation permission.

Step 3: Install the APK

Open the file manager application on the Android device, navigate to the folder containing `opti-crop.apk` (typically "Downloads"), and tap the file. The system will display an installation confirmation screen showing the app name, version, and requested permissions (the app requests only basic storage permission for saving prediction history). Tap "Install" to proceed. The installation should complete in under 30 seconds.

Step 4: Launch and Use OptiCrop

After installation, the OptiCrop icon will appear in the device's app drawer. Tap the icon to launch the application. On first launch, the app will extract the embedded machine learning model to its internal storage — this takes approximately 3–5 seconds. Once the extraction is complete, the main input form is displayed and the application is ready for offline predictions.

Step 5: Share the APK with Others

Farmers are encouraged to share the APK with neighboring farmers via Bluetooth or file transfer. The APK is self-contained and does not require an internet connection for distribution, installation, or operation.

5. Known Issues and Workarounds

The following table documents known limitations, edge cases, and their recommended workarounds. This section is maintained as a living document and is updated as new issues are identified and resolved.

#	Issue	Severity	Description	Workaround / Resolution
1	Render Cold Start Delay	Medium	On the free Render tier, the server instance spins down after 15 minutes of inactivity. The first request after a spin-down triggers a "cold start" that can take 30–60 seconds before the server responds.	Wait for the server to become responsive (up to 60 seconds) and refresh the page. For time-sensitive use, consider upgrading to a Render paid plan or use the offline Android APK.
2	Ephemeral Database on Render Free Tier	Medium	The SQLite database on Render's free tier is stored on an ephemeral filesystem that is reset during each new deployment. Logged prediction history is lost on redeploy.	For persistent storage, migrate to an external service such as AWS RDS, or use the SQLite database export feature to download a backup before each redeploy.
3	Model File Size in APK	Low	The serialized ML model is approximately 15–25 MB, contributing to the total APK size. On devices with very limited storage, this may be a concern.	The model size is inherent to the Random Forest ensemble with 200 estimators. A reduced model with fewer estimators (e.g., 50) can be generated with a minor trade-off in prediction accuracy.
4	Input Validation Range Strictness	Low	The server-side validation enforces scientifically accepted ranges for each parameter. In rare edge cases — such as soils with naturally extreme pH (below 4.0 or above 9.5) — the system will reject the input as invalid.	For exceptional soil conditions, researchers can modify the validation thresholds in the configuration to accommodate the local context.

5	Single-Crop Prediction Output	Low	The current model returns only the single top-predicted crop class. Users who want a ranked list of alternative crop options are not served by the default output.	The model supports probability-based ranking through <code>predict_proba()</code> . A future release will expose the top-three crop recommendations with probability scores.
6	No Multi-Language Support	Low	The application interface is currently available only in English, which may limit usability for farmers more comfortable in regional Indian languages (Telugu, Hindi, Marathi, etc.).	A localization roadmap is planned for a future release, starting with Telugu and Hindi. Community contributions for additional regional languages are welcomed.
7	Android Version Compatibility	Low	The APK requires Android 7.0 (Nougat, API level 24) or higher. Devices running Android 6.0 or earlier cannot install the application.	Android 7.0 was released in 2016, and the vast majority of active Android devices in India now run version 8.0 or higher. Users with older devices should use the web version via a mobile browser.
8	Concurrent Request Handling on Free Tier	Low	The free Render tier allocates limited CPU and memory resources. Under high concurrent traffic, response latency may increase or the instance may become temporarily unresponsive.	The application is designed for moderate, sequential usage patterns. For high-traffic scenarios, scale the Render instance to a paid tier or deploy locally using bootstrap scripts on multiple machines.

Table 7: Known Issues and Workarounds

Appendix: Quick Reference Card

API Endpoint Summary

Method	Endpoint	Request Body	Response	Purpose
GET	/	None	HTML page	Serves the main OptiCrop UI
POST	/predict	JSON: { nitrogen, phosphorous, potassium, ph, temperature, humidity, rainfall }	JSON: { crop, confidence, note }	Returns crop recommendation
POST	/log	JSON: { user_id, soil_id, climate_id, crop, confidence }	JSON: { status: "logged" }	Logs prediction to database (cloud only)

Table 8: API Endpoint Summary

Key File Locations

File	Path	Purpose
Flask Application	app.py	Backend server and route definitions
ML Model	model/crop_recommendation_model.pkl	Serialized prediction pipeline
Frontend Template	templates/index.html	User interface
Dependencies	requirements.txt	Python package list
Database Schema	database/schema.sql	Table creation statements
Windows Bootstrap	setup.bat	One-click setup for Windows
Linux Bootstrap	setup.sh	One-click setup for Linux/macOS
Android APK	apk/opti-crop.apk	Offline mobile application

Table 9: Key File Locations

Team Contact Matrix

Role	Name	Responsibility
Team Lead	Peddehapu Niharika	Project coordination, architecture design, documentation oversight
Member	Polupalli Sunil	Machine learning pipeline, model training, evaluation
Member	Romeo Mhakayakora	Backend development, Flask API, database design
Member	Sk Salomi	Frontend development, UI/UX design, responsive layout
Member	Jotshna Aditya Tadi	Android APK packaging, offline deployment, testing

Table 10: Team Contact Matrix